

Practical 1 - Sorting

`DSAsorts.py` implements Bubble Sort, Insertion Sort, and Selection Sort. Each algorithm uses manual index comparisons and swaps on a NumPy array.

The supplied `SortsTestHarness.py` was used unchanged for sizes 64, 128, 256, and 512. Each test was repeated three times. Times are the harness output in seconds.

The timings are machine-specific and may vary slightly when rerun.

Array Size	Arrangement	Bubble Sort	Insertion Sort	Selection Sort
64	Ascending	0.0000211	0.0000154	0.0002203
64	Descending	0.0008356	0.0005606	0.0002849
64	Random	0.0020201	0.0019495	0.0024001
128	Ascending	0.0000615	0.0000689	0.0010823
128	Descending	0.0028642	0.0028921	0.0011126
128	Random	0.0048740	0.0056087	0.0051548
256	Ascending	0.0000762	0.0000592	0.0038415
256	Descending	0.0107472	0.0091487	0.0043049
256	Random	0.0144764	0.0109371	0.0103633
512	Ascending	0.0002171	0.0001241	0.0155729
512	Descending	0.0429768	0.0368864	0.0163030
512	Random	0.0458387	0.0349765	0.0314959

Bubble Sort and Insertion Sort are fastest on ascending input. Bubble Sort has an $O(n)$ best case because it stops when a pass makes no swaps; its average and worst cases are $O(n^2)$. Insertion Sort has $O(n)$ best-case time and $O(n^2)$ average and worst cases. Selection Sort is $O(n^2)$ in all cases because it always

scans the unsorted section. The results generally increase as the array size grows, showing quadratic behaviour.